

11. Explain odds and log-odds in logistic regression. What is the cost function used in logistic regression?

Odds and Log-Odds

In Logistic Regression, we model the probability P that an event occurs.

- **Odds:** The ratio of the probability of success to the probability of failure.

$$\text{Odds} = \frac{P}{1 - P}$$

- **Log-Odds (Logit):** The natural logarithm of the odds. By taking the log of the odds, we map a probability distribution constrained between 0 and 1 to an unconstrained continuous real-valued space between $-\infty$ and $+\infty$:

$$\text{Log-Odds} = \ln \left(\frac{P}{1 - P} \right)$$

Logistic regression assumes that the log-odds of the positive class can be modeled as a linear combination of the input features:

$$\ln \left(\frac{P}{1 - P} \right) = \mathbf{w}^T \mathbf{x} + b$$

Solving this equation for P directly yields the logistic sigmoid function.

Cost Function: Binary Cross-Entropy (Log Loss)

Instead of Mean Squared Error, Logistic Regression uses Maximum Likelihood Estimation to derive its optimization cost function. For a dataset of i samples, the objective is to minimize the **Negative Log-Likelihood** (commonly called Binary Cross-Entropy or Log Loss):

$$l(\mathbf{w}) = - \sum_i [y_i \log(P_i) + (1 - y_i) \log(1 - P_i)]$$

Where $y_i \in \{0, 1\}$ is the true label and $P_i = P(y_i = 1 \mid \mathbf{x}_i, \mathbf{w})$ is the model's predicted probability. This function heavily penalizes confidently incorrect predictions.

12. Why is logistic regression considered a linear classifier?

Logistic Regression is classified as a linear classifier because its internal **decision boundary** is a **linear function** of the input features.

The model predicts probabilities using the sigmoid function:

$$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

To make a discrete class prediction, we compare this probability to a default classification threshold (typically 0.5):

$$\hat{y} = 1 \quad P(y = 1 \mid \mathbf{x}) \geq 0.5$$

Substituting the sigmoid expression shows that this condition simplifies directly to:

$$\mathbf{w}^T \mathbf{x} + b \geq 0$$

The geometric boundary separating the positive and negative predictions is defined by setting this linear combination to zero:

$$\mathbf{w}^T \mathbf{x} + b = 0$$

Because this equation defines a straight line (in 2D space), a flat plane (in 3D space), or a flat hyperplane (in higher dimensions), the classification boundaries cannot bend or curve around data points. It cuts through the feature space linearly.

13. Why is Mean Squared Error not preferred for logistic regression?

Using Mean Squared Error (MSE) to optimize a Logistic Regression model is highly problematic for two main reasons:

1. **Non-Convex Optimization Landscape:** The MSE cost function for a linear regression model is a convex "bowl" with a single global minimum. However, when you substitute the non-linear sigmoid activation function into an MSE formulation, the resulting cost function surface becomes highly **non-convex**. It gets filled with multiple local minima, plateaus, and saddle points, meaning gradient descent cannot guarantee convergence to the globally optimal parameter weights.
2. **Severe Gradient Vanishing:** The derivative of the sigmoid function $\sigma(z)$ approaches zero very rapidly as its argument z becomes highly positive or negative ($\sigma'(z) = \sigma(z)(1 - \sigma(z))$). If an MSE loss is paired with a sigmoid output, an extreme prediction error can cause the gradient to completely vanish, locking the model parameters in place and preventing learning. Binary Cross-Entropy naturally cancels out this sigmoid derivative term during differentiation, preserving healthy gradient signals throughout training.

14. What is accuracy in classification? Explain precision and recall. What is F1-score? Explain ROC curve and AUC. When is accuracy misleading? Explain false positives and false

negatives. What is class imbalance? Which metric is best for imbalanced datasets?

Confusion Matrix Elements

- **False Positive (FP):** An error where the model incorrectly predicts the positive class for a truly negative sample.
- **False Negative (FN):** An error where the model incorrectly predicts the negative class for a truly positive sample.

Performance Metrics

- **Accuracy:** The ratio of correctly predicted samples over total evaluated samples:

$$\text{accuracy} = \frac{T + TN}{T + TN + F + N}$$

- **Precision:** Out of all instances predicted as positive, how many were actually positive:

$$\text{precision} = \frac{T}{T + F}$$

- **Recall (Sensitivity):** Out of all truly positive instances in the data, how many did the model successfully identify:

$$\text{recall} = \frac{T}{T + N}$$

- **F1-Score:** The harmonic mean of precision and recall, balancing both properties into a single metric:

$$\text{F1-Score} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

ROC Curve and AUC

- **ROC Curve (Receiver Operating Characteristic):** A graphical plot showing a classifier's performance across all possible probability thresholds. It plots the True Positive Rate (Recall) on the y-axis against the False Positive Rate ($\frac{F}{TN+F}$) on the x-axis.
- **AUC (Area Under Curve):** A scalar value between 0 and 1 measuring the probability that the model will rank a randomly chosen positive instance higher than a randomly chosen negative one.

Class Imbalance and Metric Choices

Class Imbalance occurs when one class significantly outnumbers the other in the training data. Under these conditions, **accuracy becomes highly misleading**. For instance, if 99% of a medical dataset is negative, a broken model that predicts "negative" for every single patient achieves a deceptively high accuracy of 99% while missing all true positive cases.

- **Best Metrics for Imbalanced Datasets:** Precision, Recall, **F1-score**, and the **Precision-Recall AUC (PR-AUC)** curve, because they isolate the true positive class without letting the massive volume of true negatives skew the results.

15. What is the KNN algorithm? Parametric or non-parametric? Explain how KNN classification works. What is the role of K in KNN? Explain distance metrics used in KNN.

Core Architecture

The ***k*-Nearest Neighbors (KNN)** algorithm is an instance-based, lazy learning algorithm. It is strictly **non-parametric** because it does not assume any underlying mathematical distribution for the data, nor does it compress the data into a fixed set of parameter weights ; instead, it memorizes the entire training dataset to make predictions.

How KNN Classification Works

1. A new, unlabeled data point arrives in the feature space.
2. The algorithm calculates the distance between this new point and every single stored point in the training set.
3. It selects the ***K* closest training instances** based on the calculated distances.
4. It performs a **majority vote** among those ***K*** neighbors; the most frequent class label among them is assigned to the new point.

The Role of *K*

The hyperparameter ***K*** dictates the size of the neighborhood and controls the model's bias-variance tradeoff:

- **Very Small *K* (e.g., $K = 1$):** High variance and low bias. The decision boundary is highly flexible, complex, and prone to overfitting because it is heavily distorted by localized noise and outliers.
- **Very Large *K*:** Low variance and high bias. The decision boundary becomes smooth and overly simplistic, completely washing out local structural nuances (underfitting).

Distance Metrics

The choice of metric defines how proximity is calculated in the multi-dimensional feature space:

- **Euclidean Distance (L_2 Norm):** The straight-line distance between two points:

$$= \sum (x_i - y_i)^2.$$
- **Manhattan Distance (L_1 Norm):** The grid-like distance traveled along perpendicular axes:

$$= \sum |x_i - y_i|.$$
- **Minkowski Distance:** A generalized distance framework that simplifies to Manhattan when $p = 1$ and Euclidean when $p = 2$:
$$= (\sum |x_i - y_i|^p)^{1/p}.$$

16. What information can be obtained from a confusion matrix? Why is accuracy alone sometimes misleading? Explain confusion matrix for multiclass classification. How does a confusion matrix help evaluate model performance?

Information Obtained

A **confusion matrix** is a structured tabular layout displaying the exact distribution of a model's cross-referenced classification predictions against real-world grounds truths. For a binary problem, it maps out the exact counts of:

- **True Positives (TP)** and **True Negatives (TN)** (Correct predictions).
- **False Positives (FP)** (Type I errors) and **False Negatives (FN)** (Type II errors).

From these four raw counts, you can calculate all fundamental classification performance metrics, identifying exactly where and how a model is breaking down. As established in Question 14, relying solely on accuracy masks high error rates in minority classes when dealing with imbalanced datasets.

Multiclass Confusion Matrix

For an N -class classification problem, the confusion matrix expands into an $N \times N$ grid:

- **Rows** represent the **True Classes**, while **Columns** represent the **Predicted Classes** (or vice versa).
- The **main diagonal** elements (from top-left to bottom-right) represent the correct predictions for each individual class.
- Any off-diagonal element shows exactly which specific classes are being systematically confused with one another by the model. For example, a value at row `Cat` and column `Dog` indicates how many cats were mistakenly classified as dogs.

17. What are support vectors? What is the margin in SVM? What is the difference between hard margin and soft margin SVM? Why do we maximize the margin in SVM? What is the role of the regularization parameter C ?

Definitions

- **Support Vectors:** The specific training data points that lie closest to the decision boundary. They are the critical anchors of the model; if you remove them, the position of the separating hyperplane changes entirely.
- **The Margin:** The perpendicular distance between the separating decision hyperplane and the closest training data points (the support vectors).

Hard Margin vs. Soft Margin

- **Hard Margin SVM:** Assumes the training data is perfectly linearly separable. It strictly forbids any misclassifications or margin violations during training. This approach fails completely if the data contains noise or overlapping classes.
- **Soft Margin SVM:** Introduces continuous **slack variables** (ξ_i) into the mathematical optimization objective. This framework allows a controlled degree of error, permitting certain data points to cross into the margin or be misclassified to build a more robust, stable decision boundary.

Why We Maximize the Margin

Maximizing the margin reduces structural risk and improves generalization. A wider margin leaves the largest possible clearing between classes, making the model far less sensitive to minor data variations and minimizing the chance of misclassifying unseen testing data.

The Regularization Parameter C

The objective function balances margin width against margin violations:

$$\min_{\mathbf{w}, b} \frac{\|\mathbf{w}\|^2}{2} + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}\mathbf{x}_i + b) \geq 1 - \xi_i, \xi_i \geq 0$$

The hyperparameter C dictates the penalty severity for margin violations:

- **Large C :** Prioritizes minimizing classification mistakes over margin width. This can lead to a narrow margin that fits the training data tightly but generalizes poorly (**high variance / overfitting**).
- **Small C :** Prioritizes maximizing the margin width, accepting more misclassified training points to find a broader, simpler decision boundary (**high bias / underfitting**).

18. What is the kernel trick in SVM? Why do we use kernels in SVM? Explain linear kernel vs nonlinear kernel. What are common kernel functions? What is the RBF (Gaussian) kernel? How does kernel SVM handle nonlinear data?

The Kernel Trick and Handling Nonlinear Data

When a dataset is highly non-linear, it cannot be cleanly separated by a straight line or flat hyperplane in its native low-dimensional coordinate space.

Instead of mathematically calculating expensive high-dimensional coordinates explicitly, Support Vector Machines leverage the **Kernel Trick**. A kernel function computes the dot product (the similarity) between two points in a higher-dimensional feature space *implicitly*, working entirely with the original low-dimensional input vectors:

$$K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i)^T(\mathbf{x}_j)$$

This allows the SVM to find a linear, flat separating hyperplane in that high-dimensional space. When that flat boundary is projected back down into the original feature space, it forms a complex, curved non-linear decision boundary.

Linear vs. Nonlinear Kernels

- **Linear Kernel:** Computes a basic dot product in the original feature space:
 $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$. It is used when the data is already linearly separable.
- **Nonlinear Kernel:** Maps the inputs into richer, higher-dimensional spaces to discover complex, non-linear classification boundaries.

Common Kernel Functions

- **Polynomial Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + 1)^d$
- **Radial Basis Function (RBF / Gaussian) Kernel:**

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

The RBF kernel maps data into an **infinite-dimensional** feature space. It acts as a localized similarity gauge: its value approaches 1 when points are tightly clustered together, and drops to 0 when they are far apart. The hyperparameter γ controls the radius of influence for individual support vectors.

19. Write the optimization objective of SVM. What are slack variables in soft margin SVM? What is the Lagrangian formulation of SVM? Explain the constraint conditions in SVM.

Optimization Objective (Primal Form)

The primal optimization objective for a soft-margin SVM minimizes the structural norm of the weights while penalizing margin violations:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i$$

$$\text{subject to } y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i \quad \text{and} \quad \xi_i \geq 0 \quad \forall i$$

Slack Variables (ξ_i)

The **slack variables** (ξ_i) quantify the distance by which a training sample violates the ideal margin:

- $\xi_i = 0$: The point lies correctly on or outside the proper margin boundary.
- $0 < \xi_i \leq 1$: The point correctly falls on the right side of the decision boundary but lies within the margin clearing.
- $\xi_i > 1$: The point is misclassified and falls on the wrong side of the decision hyperplane.

Lagrangian Formulation (Dual Form)

To solve this constrained optimization problem, we introduce Lagrange multipliers ($\alpha_i \geq 0$ and $\mu_i \geq 0$) to construct the Lagrangian function:

$$\mathcal{L}(\mathbf{w}, b, \xi, \alpha, \mu) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i$$

Taking the partial derivatives with respect to the primal variables ($\mathbf{w}, b, \xi$) and setting them to zero yields the **Wolfe Dual formulation**:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j (\mathbf{x}_i^T \mathbf{x}_j) \\ \text{subject to} \quad & 0 \leq \alpha_i \leq C \quad \text{and} \quad \sum_{i=1}^m \alpha_i y_i = 0 \end{aligned}$$

Constraint Conditions (Karush-Kuhn-Tucker / KKT)

For the optimization solution to be valid, it must satisfy the **KKT complementary slackness conditions**:

$$\alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i] = 0$$

$$\mu_i \xi_i = (C - \alpha_i) \xi_i = 0$$

- If $\alpha_i = 0$, the point is not a support vector and has no influence on the boundary.
- If $0 < \alpha_i < C$, then $\xi_i = 0$. The point is a support vector lying exactly on the margin boundary.
- If $\alpha_i = C$, then $\xi_i > 0$. The point is a support vector that violates the margin or is misclassified.

20. What happens when data is not linearly separable? How does SVM handle outliers?

When Data is Not Linearly Separable

If the data cannot be separated by a straight line or flat hyperplane, a hard-margin SVM will fail to find a valid solution. SVM handles non-linearly separable data in two ways:

1. **Soft Margin Framework:** It uses slack variables (ξ_i) to allow a controlled number of training errors, finding the best linear compromise.
2. **The Kernel Trick:** It applies a non-linear kernel function (such as an RBF kernel) to implicitly map the features into a higher-dimensional space where a clean linear boundary can be constructed.

Handling Outliers

SVM handles anomalous outliers through its soft-margin optimization adjustments via the regularizing hyperparameter C :

- An outlier lying far inside the opposing class territory will demand a massive slack penalty value ($\xi_i \gg 1$).
- If C is configured to be **small**, the optimization loop prioritizes maximizing the margin width over correcting single anomalies. The model will simply ignore the outlier, allowing it to be misclassified to maintain a stable, well-generalized decision boundary.
- If C is configured to be **large**, the model penalizes that outlier's violation aggressively. This forces the decision boundary to bend or narrow significantly just to accommodate the outlier, which often leads to overfitting.